

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені І.І. МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій
Кафедра математичного забезпечення комп'ютерних систем

ЛАБОРАТОРНА РОБОТА №3

з дисципліни
«Інтелектуальний аналіз даних»

на тему:

K-Means Clustering у Excel та Python

студента 4 курсу
спеціальності: Інформаційні системи і технології

Краковський Владислав Олександрович

(Прізвище, ім'я та по батькові)

Викладач: доц. Петрушина Т.І.

(Прізвище, ім'я та по батькові)

Одеса – 2026

1. Постановка задачі:

Виконати кластеризацію даних за допомогою алгоритму K-Means у Microsoft Excel та Python. Реалізувати обчислення евклідових відстаней, призначення кластерів, перерахунок центроїдів та перевірку збіжності алгоритму.

2. Хід роботи:

Початкові центроїди

На першому аркуші були задані початкові центроїди кластерів.

Для кожного кластера задавалися значення семи генів.

Для кожного пацієнта обчислювалися відстані до всіх чотирьох центроїдів.

У Excel використовувалася формула:

=КОРЕНЬ(СУММКВРАЗН(\$B2:\$H2;L\$2:L\$8))

де:

- B2:H2 — значення генів пацієнта;
- L2:L8 — координати центроїда.

Функція СУММКВРАЗН обчислювала суму квадратів різниць між координатами.

Функція КОРЕНЬ обчислювала квадратний корінь.

Після цього формула копіювалася:

- вправо для всіх кластерів;
- вниз для всіх пацієнтів.

У результаті була отримана таблиця відстаней.

Призначення кластерів

Після обчислення відстаней кожен пацієнт відносився до найближчого кластера.

Для цього використовувалась формула:

=ИНДЕКС(\$Q\$1:\$T\$1;1;ПОИСКПОЗ(МИН(Q2:T2);Q2:T2;0))

де:

- МИН знаходила мінімальну відстань;
- ПОИСКПОЗ знаходила позицію мінімального значення;
- ИНДЕКС повертала назву кластера.

У результаті кожному пацієнту був призначений кластер C1–C4.

Перерахунок центроїдів

Після призначення кластерів обчислювалися нові центроїди. Новий центроїд визначався як середнє значення всіх об'єктів кластера. Для цього використовувалась функція:

=СРЗНАЧЕСЛИМН(...)

Для автоматизації посилок використовувалися:

- ДВССЫЛ
- АДРЕС
- ЛИСТ
- оператор &

Після цього створювався новий аркуш:

- лист 2;
- лист 3;
- лист 4 і т.д.

Кожен новий лист використовував результати попередньої ітерації.

Перевірка збіжності

Для перевірки завершення алгоритму використовувався параметр convergence.

На кожній ітерації порівнювалися:

- старі кластери;
- нові кластери.

Для цього використовувалась формула:

=ЕСЛИ(U3=V3;0;1)

де:

- 0 — кластер не змінився;
- 1 — кластер змінився.

Після цього обчислювалася сума всіх змін:

=СУММ(W3:W156)

Якщо convergence = 0, алгоритм вважався завершеним.

У результаті після декількох ітерацій було отримано convergence = 0.

Це означало, що кластери перестали змінюватися.

1. Реалізація множинної лінійної регресії у Python

Для реалізації моделі використовувались бібліотеки NumPy та Pandas.

```
[29]: # Импорт и чтение файла
```

```
[30]: import pandas as pd
import numpy as np

file_path = "Chapter3_dataset.xlsx"

# читаем первый лист
df_raw = pd.read_excel(file_path, sheet_name="1")

df_raw.head()
```

```
[30]:
```

	Patients- GeneSymbol	ELMO2	CREB3L1	RPS11	PNMA1	MMP2	C10orf90	ZHX3	Unnamed: 8	Unnan
0	P1	-0.840833	1.33300	1.475500	-1.39075	0.027833	-2.54950	-1.242000	NaN	
1	P2	0.018000	0.76250	0.324500	-1.48725	-0.206500	-2.76575	-0.303000	NaN	
2	P3	0.354917	0.13475	0.634000	-0.83050	0.066667	-2.76375	-0.013167	NaN	
3	P4	-0.203750	2.49875	0.483125	-0.46325	2.156000	-2.48225	-0.063500	NaN	
4	P5	0.689000	1.15725	0.952625	-2.22300	0.150167	-2.11075	-0.609000	NaN	

```
[31]: # Отделяем данные пациентов
```

```
[32]: # признаки генов
gene_columns = [
    "ELMO2",
    "CREB3L1",
    "RPS11",
    "PNMA1",
    "MMP2",
    "C10orf90",
    "ZHX3"
]

patients = df_raw["Patients-GeneSymbol"]
X = df_raw[gene_columns].copy()

X.head()
```

```
[32]:
```

	ELMO2	CREB3L1	RPS11	PNMA1	MMP2	C10orf90	ZHX3
0	-0.840833	1.33300	1.475500	-1.39075	0.027833	-2.54950	-1.242000
1	0.018000	0.76250	0.324500	-1.48725	-0.206500	-2.76575	-0.303000
2	0.354917	0.13475	0.634000	-0.83050	0.066667	-2.76375	-0.013167
3	-0.203750	2.49875	0.483125	-0.46325	2.156000	-2.48225	-0.063500
4	0.689000	1.15725	0.952625	-2.22300	0.150167	-2.11075	-0.609000

```
[33]: # Начальные центроиды
```

```
[34]: initial_centroids = pd.DataFrame(  
    {  
        "C1": [-5, 3, 1, -2, 0, -4, 3],  
        "C2": [1, -3, 3, -2, -3, 1, 2],  
        "C3": [2, -3, 3, 2, -4, -1, -3],  
        "C4": [2, 1, -1, -4, -1, -5, 2],  
    },  
    index=gene_columns  
)  
  
initial_centroids
```

```
[34]:
```

	C1	C2	C3	C4
ELMO2	-5	1	2	2
CREB3L1	3	-3	-3	1
RPS11	1	3	3	-1
PNMA1	-2	-2	2	-4
MMP2	0	-3	-4	-1
C10orf90	-4	1	-1	-5
ZHX3	3	2	-3	2

```
[35]: # Функции K-Means
# Алгоритм делает 4 вещи:

# считает расстояния до центров кластеров,
# выбирает ближайший кластер,
# пересчитывает центры,
# повторяет это, пока ничего не меняется.

[36]: # Считает расстояние от каждого пациента до каждого центра кластера.
def compute_distances(X, centroids):
    distances = pd.DataFrame(index=X.index)

    for cluster_name in centroids.columns:
        centroid_values = centroids[cluster_name].values
        # sqrt (x1-c1)^2+(x2-c2)^2+...+(xn-cn)^2
        distances[cluster_name] = np.sqrt(((X.values - centroid_values) ** 2).sum(axis=1))

    return distances

# Выбирает минимальное расстояние.
def assign_clusters(distances):
    # "Найди название столбца, где минимальное значение"
    return distances.idxmin(axis=1)

# Пересчитывает центр каждого кластера.
def update_centroids(X, clusters, cluster_names):
    new_centroids = pd.DataFrame(index=X.columns)

    for cluster_name in cluster_names:
        cluster_points = X[clusters == cluster_name]

        if len(cluster_points) == 0:
            new_centroids[cluster_name] = 0
        else:
            new_centroids[cluster_name] = cluster_points.mean(axis=0) # считает среднее по колонкам

    return new_centroids

# сколько пациентов поменяли кластер.
def count_differences(old_clusters, new_clusters):
    if old_clusters is None:
        return len(new_clusters)

    return (old_clusters != new_clusters).sum()
```

```
[37]: # Запуск алгоритма
```

```
[38]: centroids = initial_centroids.copy()
old_clusters = None

history = []

for iteration in range(1, 100):
    distances = compute_distances(X, centroids) # Считаем расстояния
    clusters = assign_clusters(distances) # Назначаем кластеры
    difference = count_differences(old_clusters, clusters) # Считаем изменения

    history.append({
        "iteration": iteration,
        "difference": difference,
        "centroids": centroids.copy(),
        "clusters": clusters.copy(),
        "distances": distances.copy()
    })

    print(f"Iteration {iteration}: difference = {difference}")

    if difference == 0: # Если изменений нет: Алгоритм останавливается.
        break

    # Пересчитываются новые центры.
    centroids = update_centroids(X, clusters, initial_centroids.columns)
    old_clusters = clusters.copy()
```

```
Iteration 1: difference = 155
Iteration 2: difference = 46
Iteration 3: difference = 18
Iteration 4: difference = 12
Iteration 5: difference = 7
Iteration 6: difference = 7
Iteration 7: difference = 7
Iteration 8: difference = 8
Iteration 9: difference = 6
Iteration 10: difference = 4
Iteration 11: difference = 8
Iteration 12: difference = 5
Iteration 13: difference = 2
Iteration 14: difference = 2
Iteration 15: difference = 0
```

```
[39]: # Финальные кластеры пациентов
```

```
[40]: result = pd.DataFrame({  
    "Patient": patients,  
    "Cluster": clusters  
})  
  
result.head(20)
```

```
[40]:
```

	Patient	Cluster
0	P1	C4
1	P2	C4
2	P3	C4
3	P4	C3
4	P5	C4
5	P6	C2
6	P7	C3
7	P8	C4
8	P9	C2
9	P10	C1
10	P11	C2
11	P12	C1
12	P13	C4
13	P14	C1
14	P15	C2
15	P16	C2
16	P17	C2
17	P18	C2
18	P19	C2
19	P20	C2

```
[41]: # Скільки пацієнтів в кожному кластері
```

```
[42]: cluster_counts = result["Cluster"].value_counts().sort_index()  
cluster_counts
```

```
[42]: Cluster  
C1      24  
C2      49  
C3      29  
C4      53  
Name: count, dtype: int64
```

```
[43]: # Фінальні центроїди
```

```
[44]: final_centroids = history[-1]["centroids"]  
final_centroids
```

```
[44]:
```

	C1	C2	C3	C4
ELMO2	-0.566444	-0.487260	-0.495408	-0.177229
CREB3L1	2.089344	0.806566	1.652095	1.131920
RPS11	0.879120	1.116194	0.700858	0.878250
PNMA1	-1.279958	-2.150000	-0.213112	-1.478208
MMP2	-0.950382	-1.048007	0.268868	-0.039626
C10orf90	-2.126031	-2.101369	-2.386810	-2.187392
ZHX3	-0.876646	-0.442088	-0.827839	-0.405327

```
[45]: # Сохранение результата в Excel
```

```
[46]: output_file = "Chapter3_HW_KMeans_Result.xlsx"  
  
with pd.ExcelWriter(output_file) as writer:  
    result.to_excel(writer, sheet_name="Final clusters", index=False)  
    cluster_counts.to_excel(writer, sheet_name="Cluster counts")  
    final_centroids.to_excel(writer, sheet_name="Final centroids")  
  
print(f"Saved to {output_file}")  
  
Saved to Chapter3_HW_KMeans_Result.xlsx
```

Результат прогнозування у Python збігся з результатом, який був отриманий у Excel.

Висновки щодо роботи

У результаті виконання лабораторної роботи було вивчено алгоритм кластеризації K-Means та реалізовано його у Microsoft Excel і Python.

Було отримано навички:

- обчислення евклідових відстаней;
- кластеризації даних;
- роботи з центроїдами;
- перерахунку центрів кластерів;
- перевірки збіжності алгоритму;
- реалізації алгоритмів інтелектуального аналізу даних.

Отримані результати показали ефективність алгоритму K-Means для групування схожих об'єктів та аналізу багатовимірних даних.

Алгоритм успішно досяг збіжності та сформував стабільні кластери пацієнтів на основі значень експресії генів.